Laboratorio Expresiones regulares

Las expresiones regulares son un medio para describir patrones de texto. Imaginemos que no sólo queremos buscar en un texto todas las líneas que contienen una palabra, como por ejemplo Barcelona, sino que sólo nos interesan las líneas que empiezan por la palabra Barcelona, pero no las que contengan la palabra en cualquier otra posición. Describir el patrón "Barcelona" es trivial, tan sólo hay que escribir "Barcelona", pero ¿cómo podemos describir "La línea comienza por la palabra Barcelona". Las expresiones regulares permiten describir este tipo de patrones de texto y muchos más por lo que nos serán de una gran utilidad. Además en Unix las expresiones regulares tienen un amplio soporte, tanto en las herramientas de procesamiento de ficheros de texto (*grep*), o en los editores de texto (*vi*, *emacs*) como en los lenguajes de programación (*Perl*, *Python*). El único inconveniente de las expresiones regulares es que su sintaxis no es trivial y que además varía ligeramente entre distintas herramientas.

En la web hay varios sitios en los que se pueden probar las expresiones regulares, como por ejemplo: Quanetic o Roblocher.

Las expresiones regulares se evalúan carácter a carácter. Las más básicas son simplemente una lista de letras que forman una texto que debe coincidir exactamente con lo que buscamos. Por ejemplo con la expresión "Human" sólo coincidirá "Human". Pero por fortuna las expresiones regulares nos permiten hacer búsquedas y substituciones mucho más complejas.

Expresiones alternativas

Una expresión u otra. Imaginemos que tenemos un fichero con los orígenes de los pacientes de un estudio sobre síndrome de Usher. Queremos seleccionar todos los pacientes que vienen de Valencia o Castellón. Podemos hacerlo utilizando la sintaxis para expresiones alternativas:

```
$ grep -E 'Valencia|Castellón' origenes.txt
Valencia        Kevin   Valencia
Castellón       Didac   valencia
Castellón       Joan    Barcelona
```

La barra vertical separa las expresiones alternativas. En este caso significa que la palabra encontrada puede ser Valencia o Castellón.

Contenedores

En muchas ocasiones nos interesa seleccionar patrones que pueden tener en una posición varias letras distintas. Por ejemplo podríamos describir el patrón "comienza por Usher y después tiene un número" o "comienza por usher y después tiene un par de letras". Busquemos todas las líneas que tienen la palabra Usher o usher:

```
$ grep -E --color '[uU]sher' origenes.txt
Hospital        nombre paciente Biobanco        Tipo Usher
Barcelona       Pepe    Barcelona       usher2A
```

Al poner las letras entre corchetes indicamos que el carácter que aparezca en esa posición puede ser cualquiera de los caracteres indicados. En este caso la expresión coincidente podría ser Usher u usher.

Utilizando esta técnica podemos también indicar rangos de caracteres, como por ejemplo “cualquier dígito entre 0 y 2”:

```
$ grep -E --color '[uU]sher[0-2]' origenes.txt
Barcelona       Pepe    Barcelona       Usher2A
Mallorca        Juan    Barcelona       usher1b
```

Existen rangos y tipos de caracteres predefinidos que podemos utilizar como:

POSIX	ASCII	Significado
[:alnum:]	[A-Za-z0-9]	Caracteres alfanuméricos (letras y números)
[:word:]	[A-Za-z0-9_]	Caracteres alfanuméricos y “_”
[:alpha:]	[A-Za-z]	Caracteres alfabéticos
[:blank:]	[\t]	Espacio y tabulador
[:space:]	[\t\r\n\v\f]	Espacios
[:digit:]	[0-9]	Dígitos
[:lower:]	[a-z]	Letras minúsculas
[:upper:]	[A-Z]	Letras mayúsculas
[:punct:]	[][!”#$%&’()*+,./:;<=>?@\^_`{|}~-]	Caracteres de puntuación

Por ejemplo, podríamos seleccionar cualquier dígito o letra:

```
$ grep --color -E '[uU]she[[:alpha:]]' origenes.txt
```

Si nos da igual que en una posición haya un carácter u otro podemos utilizar un punto “.”:

```
$ grep --color -E '.sher' origenes.txt
Hospital        nombre paciente Biobanco        Tipo Usher
Mallorca        Juan    Barcelona       usher1b
```

Dentro de los contenedores resultan también bastante útiles las negaciones. Podemos, por ejemplo, seleccionar cualquier carácter que no sea un 2 o un 3 añadiendo “^” tras el primer corchete:

```
$ grep --color -E 'sher[^23]' origenes.txt
Mallorca        Juan    Barcelona       usher1b
madrid  Alcia   madrid  usher1a
```

Cuantificadores

Además de indicar qué caracteres queremos permitir podemos seleccionar cuantas veces deben aparecer. Si no añadimos nada que indique lo contrario se asume que el carácter debe aparecer una vez, pero podríamos pedir que el carácter aparezca un número distinto de veces:

- “?”, el carácter aparece ninguna o una vez. “usher1?” coincidiría con “usher” o “usher1”.
- “*”, cero, una o varias veces.
- “+”, al menos una vez.
- “{4}”, cuatro veces.
- “{4,10}”, entre 4 y 10 veces

```
$ grep --color -E 'sher2.?' origenes.txt
Barcelona       Pepe    Barcelona       Usher2A
Albacete        Alfonso Madrid  usher2
```

Puntos de anclaje

Además de poder indicar qué y cuántas veces queremos que algo aparezca podemos indicar dónde deseamos que lo haga. Los puntos de anclaje más utilizados son:

- “^”, inicio de línea
- “$”, fin de línea
- “<”, principio de palabra
- “>”, fin de palabra
- “b”, límite de palabra

Líneas que comienzan por B:

```
$ grep --color -E '^B' origenes.txt
Barcelona       Pepe    Barcelona       Usher2A
```

Líneas que terminan por B:

```
$ grep --color -E 'B$' origenes.txt
Valencia        Kevin   Valencia        usher1B
Castellón       Didac   valencia        usher2B
```

Substituciones

Las expresiones regulares además de ser útiles para buscar patrones sirven para substituirlos. Por ejemplo podemos substituir madrid por Madrid con el siguiente comando:

```
$ sed -e 's/madrid/Madrid/' origenes.txt
Albacete        Alfonso Madrid  usher2
Madrid  Alcia   madrid  usher1a
```

Ya vimos que si queríamos que la expresión se utilizase más de una vez en cada línea debíamos utilizar el modificador “g”:

```
$ sed -e 's/madrid/Madrid/g' origenes.txt
Albacete        Alfonso Madrid  usher2
Madrid  Alcia   Madrid  usher1a
```

Estas substituciones además de soportar los patrones sencillos soportan todo lo que hemos visto anteriormente. Por ejemplo, podemos eliminar la primera columna del fichero con el comando:

```
$ sed -re 's/^[[:alnum:]]*\t//' origenes.txt
nombre paciente Biobanco        Tipo Usher
Pepe    Barcelona       Usher2A
```

Si lo que queremos es guardar el patrón encontrado para utilizarlo en la substitución debemos utilizar paréntesis. Los paréntesis indican que lo que hay dentro de ellos debe ser recordado para poder utilizado en la substitución. Por ejemplo, podríamos permutar las columnas 1 y 2 del fichero con el comando:

```
$ sed -re 's/^([[:alnum:]]*)\t([[:alnum:]]*)/\2\t\1/' origenes.txt
```

Escapes

Algunos caracteres tienen significados especiales, como ., $, (,), [,], \ o ^ y si se quieren utilizar hay que escaparlos precediéndolos con .

El campo de las expresiones regulares es muy amplio y esta pequeña introducción sólo ha pretendido mostrar algunas de las posibilidades de esta gran herramienta. El mejor modo de aprender a utilizarlas es intentar hacer uso de ellas en problemas concretos y echar una ojeada a algunos de los libros y tutoriales que se han escrito sobre ellas.

Ejercicios

1. Buscar las líneas en las que aparece la palabra bash en el archivo /etc/passwd.
2. Buscar en el archivo /etc/group todas las líneas que empiezan por *m*.
3. En el fichero anterior imprimir todas las líneas que no empiezan por *m*.
4. ¿Cuántos ficheros README hay en los subdirectorios de /usr/share/doc?
5. ¿Qué ficheros o directorios en /etc continen un número en el nombre?

Trabajando con el fichero de los orígenes de los pacientes de Usher resolver las siguientes cuestiones.

6. Seleccionar los pacientes enviados por el hospital de Castellón.
7. Seleccionar los que vienen del Biobanco de Barcelona.
8. Buscar los pacientes que no tienen Usher3.
9. En el fichero de pacientes de Usher hay algunas líneas separadas por espacios y otras por tabuladores, cambiar todos los separadores a comas.
10. Disponemos de un fichero con una caracterización morfológica de unas plantas y queremos construir un nuevo fichero que incluya solo las filas sin datos faltantes. El contenido del fichero es:

```
plant1 1.5 2.3 3.5
plant2 3.4 4.5 5.6
plant3 4.3 6.5 7.6
plant4 1.2 3.2 2.3
plant4 2.3 6.7 n.d.
plant5 4.5 4.3 2.3
plant6 4.5 4.3 2.3
plant7 3.4 - 2.3
plant8 3.4 2.3
```